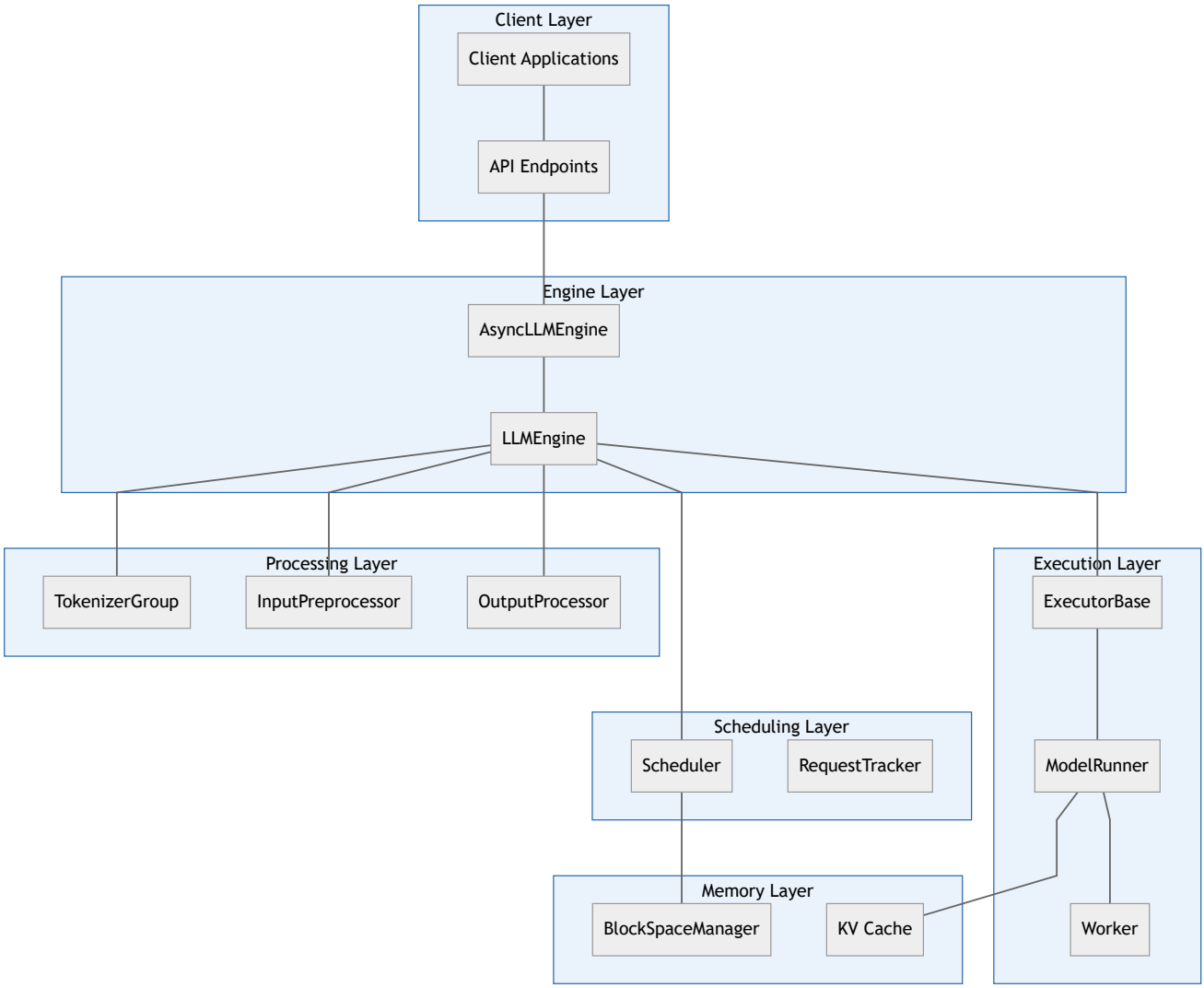
# vLLM 시스템 아키텍처 분석

## 개요

vLLM은 대규모 언어 모델의 고성능 추론 서빙을 위한 정교한 시스템 아키텍처를 가지고 있습니다. 이 문서는 vLLM의 전체 시스템 구조와 주요 구성 요소들의 역할, 그리고 이들 간의 상호작용을 상세히 분석합니다.

## 전체 시스템 아키텍처

vLLM의 시스템 아키텍처는 다음과 같은 계층적 구조로 구성됩니다:




## 주요 설계 원칙

1. **비동기 처리**: 높은 처리량을 위한 비동기 요청 처리

2. **모듈화**: 각 구성 요소의 독립적인 책임과 역할

3. **확장성**: 분산 처리와 병렬화 지원

4. **효율성**: 메모리와 연산 자원의 최적 활용

## 핵심 구성 요소 분석

### 1. LLMEngine - 메인 추론 엔진

`LLMEngine` 은 vLLM의 핵심 구성 요소로, 모든 추론 작업을 조율하는 중앙 엔진입니다.

```
class LLMEngine:
"""An LLM engine that receives requests and generates texts.

This is the main class for the vLLM engine. It receives requests
from clients and generates texts from the LLM. It includes a tokenizer, a
language model (possibly distributed across multiple GPUs), and GPU memory
space allocated for intermediate states (aka KV cache).
"""
def __init__(self, vllm_config: VllmConfig, executor_class: Type[ExecutorBase],
log_stats: bool, usage_context: UsageContext = UsageContext.ENGINE_CONTEXT):
# 설정 초기화
self.vllm_config = vllm_config
self.model_config = vllm_config.model_config
self.cache_config = vllm_config.cache_config
self.scheduler_config = vllm_config.scheduler_config
```

```
# 핵심 구성 요소 초기화
self.tokenizer = self._init_tokenizer()
self.model_executor = executor_class(vllm_config=vllm_config)
self.scheduler = [Scheduler(...) for _ in range(pipeline_parallel_size)]
self.output_processor = SequenceGroupOutputProcessor.create_output_processor(...
```

### 주요 책임

- **요청 관리**: 클라이언트 요청의 수신, 처리, 응답
- **구성 요소 조율**: 토크나이저, 스케줄러, 실행자 간의 협력 조율
- **생명주기 관리**: 시퀀스 그룹의 생성부터 완료까지 전체 생명주기 관리
- **메모리 초기화**: KV 캐시 메모리 블록 초기화 및 관리

### 핵심 메서드

```
def add_request(self, request_id: str, prompt: PromptType,
params: Union[SamplingParams, PoolingParams]) -> None:
"""요청을 엔진의 요청 풀에 추가"""
processed_inputs = self.input_preprocessor.preprocess(prompt, ...)
seq_group = self._add_processed_request(request_id, processed_inputs, params, ..
def step(self) -> List[Union[RequestOutput, PoolingRequestOutput]]:
"""한 번의 디코딩 반복 수행"""
# 1. 스케줄링
scheduler_outputs = self.scheduler.schedule()
# 2. 모델 실행
```

```
outputs = self.model_executor.execute_model(execute_model_req)
# 3. 출력 처리
request_outputs = self.output_processor.process_outputs(outputs)
return request_outputs
```

## 2. AsyncLLMEngine - 비동기 추론 엔진

`AsyncLLMEngine` 은 `LLMEngine` 을 비동기 방식으로 래핑하여 높은 동시성을 제공합니다.

```
class AsyncLLMEngine(EngineClient):
"""An asynchronous wrapper for LLMEngine.
This class uses asyncio to create a background loop that keeps
processing incoming requests. The LLMEngine is kicked by the generate
method when there are requests in the waiting queue.
"""
def  __init__(self, *args, log_requests: bool = True,
start_engine_loop: bool = True, **kwargs) -> None:
self.engine = self._engine_class(*args, **kwargs)
self._request_tracker = RequestTracker()
self.background_loop: Optional[asyncio.Future] = None
```

### LLMEngine과의 주요 차이점

| 특성 | LLMEngine | AsyncLLMEngine |

|------|----------|---------------|

| **처리 방식** | 동기식 블로킹 | 비동기 논블로킹 |

| **동시성** | 단일 요청 처리 | 다중 요청 동시 처리 |

| **사용 사례** | 배치 추론 | 온라인 서빙 |

| **응답 방식** | 완료 후 일괄 반환 | 스트리밍 응답 |

| **자원 활용** | 순차적 처리 | 백그라운드 루프 활용 |

**비동기 처리 메커니즘**

```
async def generate(self, prompt: str, sampling_params: SamplingParams,
request_id: str) -> AsyncGenerator[RequestOutput, None]:
"""비동기 생성 메서드"""
# 1. 요청 스트림 생성
stream = self._request_tracker.add_request(request_id, ...)
# 2. 백그라운드 루프 시작 (필요시)
if not self.is_running and self.start_engine_loop:
self.start_background_loop()
# 3. 스트리밍 응답 반환
async for request_output in stream.generator():
yield request_output
```

```
async def engine_step(self, virtual_engine: int) -> bool:
"""엔진 스텝 비동기 실행"""
# 새로운 요청과 중단된 요청 처리
new_requests, aborted_requests = self._request_tracker.get_new_and_aborted_reque
```

```
# 요청을 엔진에 추가
for new_request in new_requests:
await  self.engine.add_request_async(**new_request)
# 모델 스텝 실행
request_outputs = await  self.engine.step_async(virtual_engine)
# 출력을 해당 스트림에 전달
self.process_request_outputs(request_outputs)
return not all_finished
```

## 3. 요청 처리 파이프라인

vLLM의 요청 처리는 다음과 같은 단계별 파이프라인을 통해 이루어집니다:

### 3.1 요청 수신 및 전처리

```
# 1. 요청 수신
def  add_request(self, request_id: str, prompt: PromptType, params: SamplingPara
# 2. 입력 전처리
processed_inputs = self.input_preprocessor.preprocess(
prompt, lora_request=lora_request
)
# 3. 시퀀스 생성
seq = Sequence(seq_id, processed_inputs, block_size, eos_token_id)
seq_group = SequenceGroup(request_id, [seq], params)
# 4. 스케줄러에 추가
```

```
self.scheduler.add_seq_group(seq_group)
```

### 3.2 스케줄링 및 메모리 관리

```
def  schedule(self) -> SchedulerOutputs:
"""스케줄링 결정 수행"""
# 1. 실행 중인 요청 처리
running_scheduled = self._schedule_running(running_queue, budget)
# 2. 대기 중인 요청 처리
prefill_scheduled = self._schedule_prefill(waiting_queue, budget)
# 3. 메모리 블록 관리
blocks_to_swap_in, blocks_to_swap_out, blocks_to_copy = self._schedule_memory()
return SchedulerOutputs(
scheduled_seq_groups=running_scheduled + prefill_scheduled,
blocks_to_swap_in=blocks_to_swap_in,
blocks_to_swap_out=blocks_to_swap_out,
blocks_to_copy=blocks_to_copy
)
```

### 3.3 모델 실행

```
def  execute_model(self, execute_model_req: ExecuteModelRequest) -> List[Sampler
"""모델 실행 및 토큰 생성"""
# 1. 입력 준비
```

```
model_input = self._prepare_model_input(execute_model_req)

# 2. 모델 포워드 패스

hidden_states = self.model(

input_ids=model_input.input_tokens,

positions=model_input.input_positions,

kv_caches=model_input.kv_caches,

attn_metadata=model_input.attn_metadata

)

# 3. 샘플링

next_tokens = self.sampler(hidden_states, sampling_metadata)

return [SamplerOutput(outputs=next_tokens)]
```

### 3.4 출력 처리 및 응답

```
def  _process_model_outputs(self, outputs: List[SamplerOutput]) -> List[RequestO

"""모델 출력을 요청 출력으로 변환"""

request_outputs = []

for output in outputs:

# 1. 시퀀스 업데이트

self._update_sequences(output)

# 2. 완료된 시퀀스 처리

completed_seqs = self._get_completed_sequences()

# 3. 요청 출력 생성
```

```
for seq_group in completed_seqs:
request_output = RequestOutput(
request_id=seq_group.request_id,
prompt=seq_group.prompt,
outputs=[CompletionOutput(...) for seq in seq_group.seqs]
)
request_outputs.append(request_output)
return request_outputs
```

## 4. ModelRunner - 모델 실행 관리자

`ModelRunner` 는 실제 모델 실행을 담당하는 핵심 구성 요소입니다.

```
@dataclass(frozen=True)
class ModelInputForGPU(ModelRunnerInputBase):
"""GPU에서 모델 실행을 위한 입력 데이터"""
input_tokens: Optional[torch.Tensor] = None
input_positions: Optional[torch.Tensor] = None
seq_lens: Optional[List[int]] = None
attn_metadata: Optional["AttentionMetadata"] = None
multi_modal_kwargs: Optional[BatchedTensorInputs] = None
```

### 주요 책임

- **입력 준비**: 배치된 시퀀스들을 모델 입력 형태로 변환

- **메모리 관리**: GPU 메모리 할당 및 KV 캐시 관리
- **모델 실행**: 실제 신경망 포워드 패스 수행
- **출력 처리**: 모델 출력을 샘플러 출력으로 변환

### 5. TokenizerGroup - 토크나이저 관리

`TokenizerGroup` 은 LoRA 어댑터를 포함한 다양한 토크나이저를 관리합니다.

```
class TokenizerGroup:
"""A group of tokenizers that can be used for LoRA adapters."""
def __init__(self, tokenizer_id: str, enable_lora: bool, max_num_seqs: int):
self.tokenizer = get_tokenizer(tokenizer_id, **tokenizer_config)
self.lora_tokenizers = LRUCache[int, AnyTokenizer](capacity=max_loras)
def get_lora_tokenizer(self, lora_request: Optional[LoRARequest]) -> AnyTokeniz
"""LoRA 요청에 맞는 토크나이저 반환"""
if not lora_request or not self.enable_lora:
return self.tokenizer
if lora_request.lora_int_id not in self.lora_tokenizers:
tokenizer = get_lora_tokenizer(lora_request, **self.tokenizer_config)
self.lora_tokenizers.put(lora_request.lora_int_id, tokenizer)
return self.lora_tokenizers[lora_request.lora_int_id]
```

## 비동기 처리와 동기화 메커니즘

### 1. RequestTracker - 요청 추적 관리

```
class RequestTracker:

"""Synchronous abstraction for tracking requests."""

def  __init__(self) -> None:

self._request_streams: Dict[str, AsyncStream] = {}

self._aborted_requests: asyncio.Queue[str] = asyncio.Queue()

self._new_requests: asyncio.Queue[Tuple[AsyncStream, dict]] = asyncio.Queue()

self.new_requests_event = asyncio.Event()

def  add_request(self, request_id: str, **kwargs) -> AsyncStream:

"""비동기 스트림을 통한 요청 추가"""

stream = AsyncStream(request_id, self.abort_request)

self._new_requests.put_nowait((stream, {"request_id": request_id, **kwargs}))

self.new_requests_event.set()

return stream
```

## 2. AsyncStream - 비동기 스트림 처리

```
class AsyncStream:

"""A stream of RequestOutputs for a request that can be iterated over asynchrono

def  __init__(self, request_id: str, cancel: Callable[[str], None]) -> None:

self.request_id = request_id

self._cancel = cancel

self._queue: asyncio.Queue = asyncio.Queue()

self._finished = False
```

```
async def generator(self) -> AsyncGenerator[Union[RequestOutput, PoolingReques
"""비동기 제너레이터를 통한 스트리밍 출력"""
try:
while True:
result = await self._queue.get()
if result == STOP_ITERATION:
return
yield result
except GeneratorExit:
self._cancel(self.request_id)
raise asyncio.CancelledError from None
```

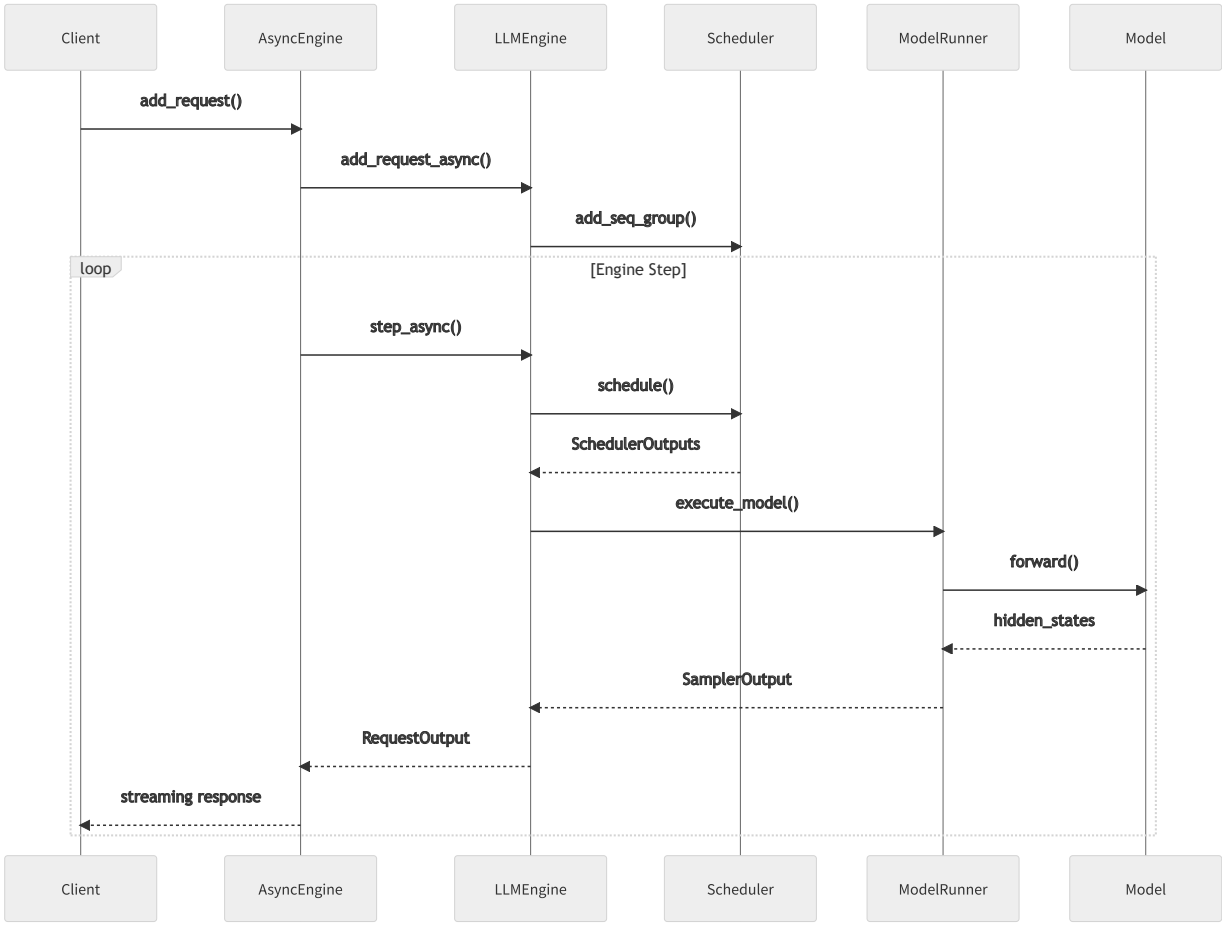
### 3. 백그라운드 루프 처리

```
async def run_engine_loop(self) -> None:
"""백그라운드에서 실행되는 메인 엔진 루프"""
has_requests_in_progress = False
while True:
if not has_requests_in_progress:
# 새로운 요청 대기
await self._request_tracker.wait_for_new_requests()
# 각 가상 엔진에 대해 스텝 실행
```

```
has_requests_in_progress = False

for virtual_engine in  range(self.parallel_config.pipeline_parallel_size):

has_requests = await  self.engine_step(virtual_engine)

has_requests_in_progress = has_requests_in_progress or has_requests

# CPU 양보

await asyncio.sleep(0)
```

## 구성 요소 간 데이터 흐름

### 1. 요청 데이터 흐름


Client
AsyncEngine
LLMEngine
Scheduler
ModelRunner
Model
add_request()
add_request_async()
add_seq_group()
loop
[Engine Step]
step_async()
schedule()
SchedulerOutputs
execute_model()
forward()
hidden_states
SamplerOutput
RequestOutput
streaming response
Client
AsyncEngine
LLMEngine
Scheduler
ModelRunner
Model

## 2. 메모리 데이터 흐름

```
# KV 캐시 블록 할당 흐름

def  allocate_blocks_for_sequence(self, seq: Sequence) -> None:

"""시퀀스를 위한 메모리 블록 할당"""

# 1. 필요한 블록 수 계산

num_required_blocks = math.ceil(seq.get_len() / self.block_size)

# 2. GPU 블록 할당 시도

if  self.gpu_allocator.get_num_free_blocks() >= num_required_blocks:

blocks = self.gpu_allocator.allocate(num_required_blocks)

seq.block_table = blocks

else:

# 3. 메모리 부족 시 스와핑 또는 선점

self._handle_memory_pressure(seq, num_required_blocks)
```

## 3. 제어 흐름

```
def  step(self) -> List[RequestOutput]:

"""메인 제어 흐름"""

# 1. 스케줄링 결정

scheduler_outputs = self.scheduler.schedule()

if not scheduler_outputs.is_empty():

# 2. 메모리 작업 수행

self._execute_memory_operations(scheduler_outputs)

# 3. 모델 실행
```

```
execute_model_req = ExecuteModelRequest(
seq_group_metadata_list=scheduler_outputs.scheduled_seq_groups,
blocks_to_swap_in=scheduler_outputs.blocks_to_swap_in,
blocks_to_swap_out=scheduler_outputs.blocks_to_swap_out,
blocks_to_copy=scheduler_outputs.blocks_to_copy
)
outputs = self.model_executor.execute_model(execute_model_req)
# 4. 출력 처리
request_outputs = self.output_processor.process_outputs(outputs)
else:
request_outputs = []
return request_outputs
```

## 확장성과 안정성을 위한 설계 원칙

### 1. 모듈화와 추상화

vLLM은 각 구성 요소를 명확한 인터페이스로 분리하여 모듈화를 달성합니다:

```
# 추상 기본 클래스를 통한 인터페이스 정의
class ExecutorBase(ABC):
"""Base class for all executors."""
@abstractmethod
def  execute_model(self, execute_model_req: ExecuteModelRequest) -> List[Sampler
```

```
raise NotImplementedError

@abstractmethod

def  collective_rpc(self, method: str, *args, **kwargs) -> List[Any]:

raise NotImplementedError



# 구체적인 구현체들

class RayDistributedExecutor(ExecutorBase): ...

class MultiprocessingDistributedExecutor(ExecutorBase): ...

class UniProcExecutor(ExecutorBase): ...
```

## 2. 설정 기반 아키텍처

```
@dataclass

class VllmConfig:

"""vLLM의 모든 설정을 포함하는 통합 설정 클래스"""

model_config: ModelConfig

cache_config: CacheConfig

parallel_config: ParallelConfig

scheduler_config: SchedulerConfig

device_config: DeviceConfig

load_config: LoadConfig

lora_config: Optional[LoRAConfig] = None

speculative_config: Optional[SpeculativeConfig] = None
```

```
decoding_config: Optional[DecodingConfig] = None

observability_config: Optional[ObservabilityConfig] = None
```

### 3. 오류 처리 및 복구

```
class AsyncEngineDeadError(RuntimeError):

"""비동기 엔진 오류 처리"""

pass


def _error_callback(self, exc: Exception) -> None:

"""오류 콜백을 통한 예외 전파"""

self.set_errored(exc)

self._request_tracker.propagate_exception(exc)


# 요청별 오류 처리

def process_exception(self, request_id: str, exception: BaseException) -> None:

"""개별 요청의 예외 처리"""

self.abort_request(request_id, exception=exception)
```

### 4. 리소스 관리

```
def __del__(self):

"""리소스 정리를 위한 소멸자"""
```

```
# 모델 실행자 종료
if model_executor := getattr(self, "model_executor", None):
model_executor.shutdown()


def shutdown_background_loop(self) -> None:
"""백그라운드 루프 정리"""
if self._background_loop_unshielded is not None:
self._background_loop_unshielded.cancel()
self._background_loop_unshielded = None
self.background_loop = None
```

## 성능 최적화 설계 패턴

### 1. 지연 초기화 (Lazy Initialization)

```
@cached_property  # RPC 호출 방지를 위한 캐싱
def supported_tasks(self) -> tuple[SupportedTask, ...]:
output = self.collective_rpc("get_supported_tasks")
return output[0]
```

### 2. 객체 풀링

```
class PyObjectCache:
```

```
"""Python 객체 캐싱을 통한 메모리 할당 최적화"""

def __init__(self, max_size: int = 1024):

self._cache: Dict[Type, List[Any]] = {}

self._max_size = max_size
```

### 3. 배치 처리 최적화

```
def _prepare_model_input(self, seq_group_metadata_list: List[SequenceGroupMetad

"""여러 시퀀스를 효율적인 배치로 결합"""

# 토큰 배치 생성

input_tokens = []

input_positions = []

for seq_group_metadata in seq_group_metadata_list:

for seq_data in seq_group_metadata.seq_data.values():

input_tokens.extend(seq_data.get_token_ids())

input_positions.extend(range(seq_data.get_len()))

return ModelInputForGPU(

input_tokens=torch.tensor(input_tokens),

input_positions=torch.tensor(input_positions),

# ... 기타 배치 데이터

)
```

## 결론

vLLM의 시스템 아키텍처는 고성능 LLM 추론 서빙을 위해 정교하게 설계된 계층적 구조를 가지고 있습니다. 주요 특징은 다음과 같습니다:

1. **비동기 처리**: `AsyncLLMEngine` 을 통한 높은 동시성 지원
2. **모듈화**: 각 구성 요소의 명확한 책임 분리와 인터페이스 정의
3. **확장성**: 분산 실행과 병렬 처리를 위한 유연한 아키텍처
4. **효율성**: 메모리 관리, 배치 처리, 캐싱을 통한 성능 최적화
5. **안정성**: 포괄적인 오류 처리와 리소스 관리

이러한 아키텍처 설계를 통해 vLLM은 기존 솔루션 대비 뛰어난 성능과 확장성을 달성하며, 다양한 배포 환경에서 안정적인 LLM 서빙 서비스를 제공할 수 있습니다.